EvoLeap Module

Reward Branch Augmentation

```
def calculate_reward(self, vehicles_status,
vehicles_status_next, action):
    reward = 0
    # Reward for forming platoons
    for car_idx, status in vehicles_status_next.items():
        initial_partners = len(vehicles_status[car_idx].get
        ('platoon_partner', []))
        new_partners = len(status.get('platoon_partner',
        []))

        # Reward for adding new members to the platoon
        if new_partners > initial_partners:
            reward += (new_partners - initial_partners) * 10

        # Slight negative reward for disbanding a platoon
        if new_partners < initial_partners:
            reward -= (initial_partners - new_partners) * 5

        # Additional reward proportional to the size of the
        platoon
        if new_partners > 0:
            reward += new_partners * 2  # Giving a small
            bonus per platoon member

    return reward
```

Add Platoon Size Reward

Reward Equilibrium Tuning

```
def calculate_reward(self, vehicles_status,
vehicles_status_next, action):
    reward = 0
    # Enhanced reward for forming platoons
    for car_idx, status in vehicles_status_next.items():
        initial_partners = len(vehicles_status[car_idx].get
        ('platoon_partner', []))
        new_partners = len(status.get('platoon_partner',
        []))

        # Increase reward for adding new members to the
        platoon
        if new_partners > initial_partners:
            reward += (new_partners - initial_partners) * 15

        # Increase penalty for disbanding a platoon to
        better disincentivize this behavior
        if new_partners < initial_partners:
            reward -= (initial_partners - new_partners) * 8

    return reward
```

Modify weights of reward

Reward Function Code Temple

```
def calculate_reward(self, vehicles_status,
vehicles_status_next, action):
    reward = 0
    # Reward for forming platoons
    for car_idx, status in vehicles_status_next.items():
        initial_partners = len(vehicles_status[car_idx].get
        ('platoon_partner', []))
        new_partners = len(status.get('platoon_partner',
        []))

        # Reward for adding new members to the platoon
        if new_partners > initial_partners:
            reward += (new_partners - initial_partners) * 10

        # Slight negative reward for disbanding a platoon
        if new_partners < initial_partners:
            reward -= (initial_partners - new_partners) * 5

    return reward
```

Reward Prune Refinement

```
def calculate_reward(self, vehicles_status,
vehicles_status_next, action):
    reward = 0
    # Reward for forming or growing platoons
    for car_idx, status in vehicles_status_next.items():
        initial_partners = len(vehicles_status[car_idx].get
        ('platoon_partner', []))
        new_partners = len(status.get('platoon_partner',
        []))

        # Reward for adding new members to the platoon
        if new_partners > initial_partners:
            reward += (new_partners - initial_partners) * 10

    return reward
```

Delete negative reward for disbanding a platoon

Reward Paradigm Leap

```
def calculate_reward(self, vehicles_status,
vehicles_status_next, action):
    reward = 0
    # Reward for maintaining platoons and penalize
    excessive delays
    for car_idx, status in vehicles_status_next.items():
        current_platoon_size = len(status.get
        ('platoon_partner', []))
        delay_time = status.get('delay_time', 0)
        delay_budget = status.get('delay_time_budget', float
        ('inf'))

        # Reward for maintaining or having a large platoon
        reward += current_platoon_size * 8

        # Penalize for delays exceeding the planned budget
        if delay_time > delay_budget:
            reward -= (delay_time - delay_budget) * 2

        # Reward for effectively managing delays within the
        budget
        else:
            reward += (delay_budget - delay_time) * 0.5

    # Normalize reward
    reward /= len(vehicles_status_next)
    return reward
```

Refactor reward function